



Module 5b

Efficient SQL

Writing SQL that performs on a Yellowbrick MPP column store — Problem · Why · Fix

Your Two Best Tools: EXPLAIN & sys.query

Don't guess. Read the plan and the metrics — the fix is almost always visible there.

EXPLAIN <query_id> / EXPLAIN (ANALYZE)

- **Plan shape** read the join order, join types, and where DISTRIBUTE / BROADCAST nodes appear.
- **read_efficiency** low % means the scan is reading data it then throws away — a skipping problem.
- **skew** actual rows per worker uneven → runtime skew.
- **Casts** watch for ::INT8 / ::VARCHAR injected into join conditions — silent type mismatches.
- **rows_planned vs actual** big gaps = bad estimates, usually from expressions or multi-column joins.

sys.query / sys.log_query

- **State & blocked** is it running, queued, or waiting on a lock?
- **run_ms / wait_run_cpu_ms** real work vs waiting for a time slice (concurrency).
- **io_spill_space_bytes_max** any spill is a red flag — typically a 10x+ penalty.
- **io_read_bytes** high reads with few rows returned = poor scan/skip.
- **rows_returned** huge result sets belong in ybunload, not a client fetch.

Filtering & Predicates

Write WHERE clauses that let Yellowbrick skip data it doesn't need to read.

Range-Restricted Scans: The Core Idea

A column store with no indexes gets its speed from NOT reading data — if your predicate allows it.

PROBLEM

- Queries scan far more data than they return.
- High `io_read_bytes` with few rows out.
- Low `read_efficiency` in `EXPLAIN (ANALYZE)`.
- Filters that 'should' be fast are slow.

WHY IT HAPPENS

- No indexes — scans rely on shard/block min-max metadata to skip data.
- Skipping (a.k.a. shard/block pruning) only works when the predicate can be compared to the stored min/max.
- If the engine can't reason about the predicate, it must read everything.

HOW TO FIX

- Write predicates the optimiser can prune on.
- Compare a column to a constant or constant expression.
- Keep data ordered (`SORT ON`) so matching rows cluster together.
- Verify skipping with `read_efficiency` in `EXPLAIN (ANALYZE)`.

What Enables (and Disables) Skipping

The rule of thumb: a bare column on one side, a constant on the other.

- **Allowed operators** Pruning works with = , != , < , <= , > , >= , IS NULL, IS NOT NULL, IN, NOT IN — compared to constants.
- **Constant side only** The cast or expression must be on the constant side, never on the scanned column.
- **Expressions on the column kill it** LOWER(type)='drop' or col::DATE forces a full scan — there is no metadata for a computed value.
- **Neither-side-constant kills it** done_time = state_time can't prune; there's no constant to compare min/max against.
- **Ordering amplifies it** Well-ordered data means matching rows sit in a few shards/blocks; unordered data is scattered everywhere.

BETWEEN vs IN

- Both can prune.
- A BETWEEN range is the most efficient.
- Large IN-lists prune best on YB 7+.

Filtering: Offending vs Fixed

EXPLAIN (ANALYZE): watch read_efficiency jump and io_read_bytes fall after the rewrite.

OFFENDING SQL

```
-- expression on the SCANNED column
-- (cannot prune -> full scan)
SELECT count(*)
FROM   log_query
WHERE  submit_time::DATE
       > current_date - 7;

SELECT count(*)
FROM   events
WHERE  LOWER(type) = 'drop';
```

The cast / function is applied to the column, so no shard or block can be skipped.

FIXED SQL

```
-- put the expression on the CONSTANT side
SELECT count(*)
FROM   log_query
WHERE  submit_time
       > current_date - 7;

SELECT count(*)
FROM   events
WHERE  type = 'DROP'; -- or store
                        -- lower-cased at load
```

Bare column vs constant expression -> min/max metadata prunes shards & blocks.

Takeaway: Keep the scanned column bare; move every cast, function, and arithmetic to the constant side.

Skipping Pays Off: Read Only What Matters

A filter on a sorted column turns a full scan into a handful of block reads.

Block 1	Jan 01 – Jan 24	skipped
Block 2	Jan 25 – Feb 16	skipped
Block 3	Feb 17 – Mar 11	✓ READ
Block 4	Mar 12 – Apr 04	✓ READ
Block 5	Apr 05 – Apr 28	skipped
Block 6	Apr 29 – May 22	skipped
Block 7	May 23 – Jun 15	skipped
Block 8	Jun 16 – Jul 09	skipped

- With ordered data and a prunable predicate, the engine reads only the shards/blocks whose min-max range can match the filter.
- Everything else is skipped before any decompression — the column store's substitute for an index.
- Lose pruning (an expression on the column, or unordered data) and you pay full-scan I/O on every query.

Joins

Collocation, join order, hash-chain skew, and the silent cost of casts and expressions.

Collocation & Redistribution

In an MPP system the network is the enemy; the best join moves no data at all.

PROBLEM

- Joins are slow despite small result sets.
- EXPLAIN shows DISTRIBUTE or BROADCAST nodes.
- High io_network_bytes during the join.

WHY IT HAPPENS

- To join, matching rows must be on the same worker (collocated).
- If the two sides aren't distributed on the join key, one or both sides are redistributed over the network.
- Replicated-to-replicated joins run on a single worker then redistribute.

HOW TO FIX

- Distribute large fact tables on the common join key; REPLICATE small dimensions.
- Join on the distribution key so no redistribution is needed.
- If a one-off join needs a different key, stage it in a CTAS/temp table distributed correctly.

Reading Redistribution in a Plan

If you see a DISTRIBUTE node under a join, ask: can these tables share a distribution key?

EXPLAIN

```
INNER JOIN ON (f.cust_sk = c.cust_sk)
|-SCAN customer AS c
|-BUILD
  DISTRIBUTE ON HASH(f.cust_sk) <-- moves
  SCAN store_sales AS f

-- vs. collocated (both on cust_sk):
INNER JOIN ON (f.cust_sk = c.cust_sk)
|-SCAN customer AS c
|-BUILD
  SCAN store_sales AS f <-- no move
```

- **DISTRIBUTE / BROADCAST** every such node is network traffic — the thing to design away.
- **Build vs probe** the smaller (build) side is hashed in memory; the larger (probe) side streams past.
- **No movement** when both sides share the distribution key, the join is local on every worker.
- **Fix in DDL or staging** change the table's distribution, or pre-stage one side with the right key.

Join Order

Filter early. Start from the fact table, reduce rows, then bring in the rows-adding joins.

PROBLEM

- Memory use and runtime balloon on multi-table queries.
- Intermediate row counts explode mid-plan.
- Worse with deeply nested subqueries.

WHY IT HAPPENS

- Each join stage processes whatever the previous stage produced.
- Doing a row-multiplying join before a row-reducing one carries huge intermediates forward.
- The planner only permutes the first 8 tables; SQL order and estimates steer the rest.

HOW TO FIX

- Put the most selective joins (those that filter most) first.
- Help estimates: constraints, materialised columns, accurate types.
- Force order with a CTE + OFFSET 0, or raise/lower join_collapse_limit (see Part 10).

Hash Joins & Long Hash Chains

Hash joins assume an even spread of values. When that assumption breaks, they degrade badly.

PROBLEM

- A single join step dominates runtime / memory.
- One worker is far busier than the rest (skew).
- Plan shows a hash join on a low-cardinality key.

WHY IT HAPPENS

- Hash joins bucket rows by hashed key; ideally one row per bucket.
- Skew, too few buckets (bad estimates), or many rows sharing a value create long chains.
- Every probe row then walks a long linked list — $O(n)$ per row.

HOW TO FIX

- Improve estimates (constraints, stats, avoid expressions).
- Raise cardinality: concatenate multi-column low-card keys; add a synthetic key.
- Handle NULL skew explicitly (next slides); consider a HINT to flip build/probe.

Skew-Specific Join Rewrites

When skew is in the data itself, reshape the query so the skewed value doesn't drive the join.

OFFENDING SQL

```
-- high-NULL outer join: all NULLs
-- collide on one worker (long chain)
SELECT ...
FROM orders o
LEFT JOIN ref r
  ON o.ref_id = r.ref_id;

-- multi-col low-card join,
-- only one col is the dist key
```

NULLs (or one repeated value) pile onto a single worker; the join serialises there.

FIXED SQL

```
-- split NULL vs non-NULL, UNION ALL
SELECT ... FROM orders o
JOIN ref r ON o.ref_id = r.ref_id
WHERE o.ref_id IS NOT NULL
UNION ALL
SELECT ... FROM orders o
WHERE o.ref_id IS NULL;

-- or join on a concatenated key
```

NULL rows bypass the hash join; matched rows spread evenly. Concatenate low-card keys to raise cardinality.

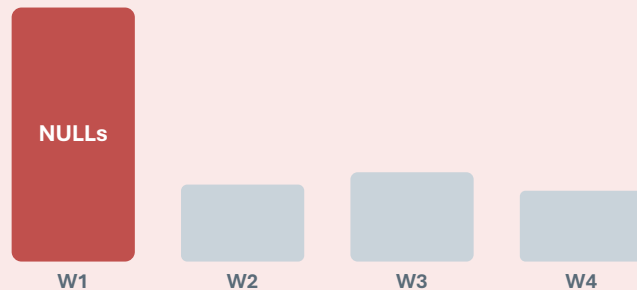
Takeaway: For NULL/low-cardinality skew: separate the skewed value with UNION ALL, concatenate multi-column keys, or HINT the build side.

Why the UNION ALL Skew Rewrite Works

Peeling the skewed value out of the join removes the single-worker hotspot.

BEFORE — ONE HOT WORKER

All NULLs hash to the same worker -> a long hash chain on one node. And NULL never equals NULL, so those rows never match anyway.



AFTER — SPLIT WITH UNION ALL

WHERE ref_id IS NOT NULL -> real join, spread evenly



WHERE ref_id IS NULL -> no join; emit rows with NULLs on the right.

UNION ALL stitches the two disjoint sets back together — and because the branches can't overlap, no dedupe is needed (that's why UNION ALL, not UNION).

Takeaway: The join only processes matchable, evenly-spread rows; the skewed NULLs bypass it entirely.

Joins on Expressions & Implicit Casts

EXPLAIN reveals it: look for `::INT8` / `::VARCHAR` and a `DISTRIBUTE ON HASH(expr)` under the join.

OFFENDING SQL

```
-- mismatched types -> implicit cast
CREATE TABLE t1 (id4 INT4) DISTRIBUTE(id4);
CREATE TABLE t2 (id8 INT8) DISTRIBUTE(id8);
SELECT *
FROM t1 JOIN t2
  ON t1.id4 = t2.id8;  -- id4::INT8

-- function in the join key
SELECT * FROM a JOIN b
  ON UPPER(a.code) = UPPER(b.code);
```

Casts/expressions in the ON clause force redistribution and wreck row estimates.

FIXED SQL

```
-- match the types in the model
-- (or cast the SMALL/constant side only)
SELECT *
FROM t1 JOIN t2
  ON t1.id8 = t2.id8;

-- materialise the expression as a column
-- code_uc = UPPER(code) at load time
SELECT * FROM a JOIN b
  ON a.code_uc = b.code_uc;
```

Same types + bare columns -> collocated, well-estimated join.

Takeaway: Keep join keys the same type and free of functions; materialise any needed expression as a real column.

Aggregations & GROUP BY

Group on the right columns, in the right order — and rewrite COUNT(DISTINCT).

GROUP BY on the Distribution Column

Same query, two columns — the difference is entirely the redistribution cost.

OFFENDING SQL

```
-- group on a NON-distribution column
-- (store_sales distributed on cust_sk)
SELECT count(*)
FROM (
  SELECT ss_addr_sk
  FROM   store_sales
  GROUP BY 1
) q;
-- ~39s : rows redistributed first
```

Grouping on a non-distribution column redistributes every row across the network before aggregating.

FIXED SQL

```
-- group on the distribution column
SELECT count(*)
FROM (
  SELECT ss_customer_sk
  FROM   store_sales
  GROUP BY 1
) q;
-- ~24s : aggregation is local
-- (best fix: distribute on the
-- column you group/join on most)
```

Local aggregation, no shuffle. Choose the table's distribution key to match the heaviest GROUP BY / join.

Takeaway: Aggregate on the distribution key where you can; otherwise expect (and plan for) a redistribution.

Low-Cardinality Aggregation Skew

The engine distributes aggregations on a hash of the leading grouping columns — order matters.

PROBLEM

- Aggregations with many GROUP BY columns are slow/skewed.
- Especially wide GROUP BYs over big fact tables.
- One worker does most of the aggregation work.

WHY IT HAPPENS

- Redistribution hashes only the FIRST 8 GROUP BY columns.
- If those 8 are all low-cardinality, the hash has very few distinct values.
- Rows pile onto a handful of workers -> runtime skew.

HOW TO FIX

- Order the GROUP BY so the highest-cardinality columns come first.
- Add/Use a composite key with enough distinct values.
- For COUNT(DISTINCT), use the double-grouping rewrite (next slide).

COUNT(DISTINCT): Double-Grouping Rewrite

A classic MPP rewrite — see Support Article - 'Rewriting COUNT DISTINCT Queries for Better Performance'.

OFFENDING SQL

```
-- one pass, heavy redistribution per
-- DISTINCT; intermediate recomputed
SELECT group_col,
       COUNT(DISTINCT user_id)
FROM   events_v
GROUP BY group_col;
```

COUNT(DISTINCT) redistributes all data per distinct expression and recomputes the source set.

FIXED SQL

```
-- group to distinct pairs first,
-- then a cheap COUNT(*)
SELECT group_col, COUNT(*)
FROM (
  SELECT group_col, user_id
  FROM   events_v
  GROUP BY group_col, user_id
) q
GROUP BY group_col;
-- alt: CTE + OFFSET 0 on the source
```

Two cheap GROUP BYs replace one expensive distinct shuffle — often dramatically faster.

Takeaway: Rewrite COUNT(DISTINCT) as group-then-count; if the source is a view/subquery, pin it with a CTE + OFFSET 0.

CTEs, Subqueries & Materialization

Use CTEs and temp tables to control join order, reuse, and statistics.

CTEs Are Not Materialized

A CTE is great for readability and order control — but it doesn't cache results by default.

PROBLEM

- A CTE referenced multiple times is recomputed each time.
- Planner picks a poor join order across CTE boundaries.
- Repeated expressions evaluated again and again.

WHY IT HAPPENS

- In Yellowbrick a CTE is inlined, not materialized — no virtual table, no statistics.
- So the optimiser may reorder or duplicate work you expected to compute once.
- (PostgreSQL's MATERIALIZE keyword is not supported.)

HOW TO FIX

- Force materialization with OFFSET 0 (or a large LIMIT) to pin order.
- For reuse + statistics, create a TEMP table (CTAS) instead.
- Pull repeated subqueries into one CTE/temp table.

OFFSET 0 to Force Join Order

To return the first row of the CTE, the whole inner result must be built first — that's the lever.

OFFENDING SQL

```
-- planner free to reorder a,b,c
SELECT *
FROM a
JOIN b ON a.id = b.a_id
JOIN c ON b.id = c.b_id;
-- bad order on skewed/expr joins
```

When estimates are poor, the planner may pick a costly order across the joins.

FIXED SQL

```
-- materialise (a JOIN b) first
WITH ab AS (
  SELECT *
  FROM a JOIN b ON a.id = b.a_id
  OFFSET 0      -- forces it
)
SELECT *
FROM ab JOIN c ON c.b_id = ab.id;
```

OFFSET 0 forces the inner result to be produced first, pinning the join order you want.

Takeaway: Use a CTE + OFFSET 0 (or large LIMIT) to force join order when the planner's estimates are unreliable.

CTAS / TEMP Table for Stats & Re-Distribution

CTAS is the Swiss-army fix for poor estimates, repeated work, and mismatched distribution.

OFFENDING SQL

```
-- repeated expensive subquery,  
-- wrong distribution for the join,  
-- poor estimates from expressions  
SELECT ...  
FROM big f  
JOIN (/ * heavy derived set */) d  
  ON f.k = d.k      -- redistributes  
WHERE ...          -- multiple times
```

Derived sets have no stats and may carry the wrong distribution into a big join.

FIXED SQL

```
CREATE TEMP TABLE d AS  
  SELECT ... /* heavy set */  
  DISTRIBUTE ON (k);    -- right key  
-- stats now exist on d  
SELECT ...  
FROM big f JOIN d ON f.k = d.k;
```

A temp table gets statistics, the distribution you choose, and is computed once.

Takeaway: Materialise to a TEMP table when you need a different distribution key, better estimates, or to compute once.

Data Modification (DML)

UPDATE, DELETE, TRUNCATE and DROP behave very differently on a column store.

Singleton UPDATE / DELETE

The single biggest CDC/ELT mistake on Yellowbrick is treating it like an OLTP database.

PROBLEM

- ELT/CDC jobs run thousands of single-row UPDATE/DELETE.
- Blocking locks and slow commits.
- GC backlog and front-end pressure.

WHY IT HAPPENS

- Each statement needs an exclusive lock and a backend commit coordinated across all workers.
- UPDATE = delete + re-insert; every change tracks a deleted row.
- Row-at-a-time fights the set-based engine.

HOW TO FIX

- Stage changes in a temp table, then a SINGLE set-based UPDATE/DELETE/MERGE.
- Avoid long-running open transactions.
- Commit & close connections promptly (watch psycopg/.NET autocommit).

Set-Based UPDATE / DELETE with a Staging Table

Collect the change set in a temp table, then apply it with one UPDATE ... FROM and one DELETE ... USING.

OFFENDING SQL: ROW-BY-ROW

```
-- ELT/CDC tool loops: one statement
-- per changed row -> N locks + N commits
UPDATE fact_sales
SET   payment_status = 'refunded'
WHERE transaction_id = :id;  -- repeated

DELETE FROM fact_sales
WHERE transaction_id = :id;  -- repeated
```

Every row is its own transaction: an exclusive lock, a cross-worker commit, and delete-tracking. The manager node saturates.

STAGED + SET-BASED

```
-- 1) stage every change once (bulk-loaded),
--    collocated on the join key
CREATE TEMP TABLE chg
(transaction_id varchar(50), new_status varchar(20))
DISTRIBUTE ON (transaction_id) ON COMMIT DROP;

-- 2) apply all at once: UPDATE ... FROM
UPDATE fact_sales f
SET   payment_status = c.new_status
FROM   chg c
WHERE  f.transaction_id = c.transaction_id;

-- 3) and DELETE ... USING
DELETE FROM fact_sales f
USING   chg c
WHERE   f.transaction_id = c.transaction_id;
```

One lock, one commit, run in parallel across workers. Distribute chg on the join key so the apply stays local.

Takeaway: Turn N row-at-a-time round trips into one collocated, single-commit, set-based statement. Yellowbrick has no MERGE — use UPDATE ... FROM and DELETE ... USING.

Large UPDATE / DELETE: Recreate Instead

An UPDATE is a delete plus an insert; at scale, rebuilding is cheaper and cleaner.

OFFENDING SQL

```
-- rewrites + delete-tracks millions
-- of rows; huge I/O, GC, BAR load
UPDATE store_sales
SET status = 'CLOSED'
WHERE fiscal_year < 2020;

DELETE FROM store_sales
WHERE fiscal_year < 2010;
```

Large UPDATE/DELETE generate massive delete-tracking, GC, and replication/backup overhead.

FIXED SQL

```
-- rebuild the table you want
CREATE TABLE store_sales_new AS
SELECT *,
       CASE WHEN fiscal_year < 2020
            THEN 'CLOSED' ELSE status END
FROM store_sales
WHERE fiscal_year >= 2010
DISTRIBUTE ON (ss_customer_sk)
ORDER BY ss_sold_date_sk;
-- swap names
```

A single bulk append (CTAS) avoids per-row delete tracking and produces clean, ordered shards.

Takeaway: For large-scale modifications, recreate the table via CTAS rather than UPDATE/DELETE-ing in place.

DELETE vs TRUNCATE vs DROP — and Temp Tables

Every DELETE/TRUNCATE leaves delete-tracking behind; choose the lightest operation that does the job.

1

Prefer DROP over TRUNCATE for transient tables

DROP avoids delete-tracking entirely and reduces replication / incremental-backup size. TRUNCATE is an optimised DELETE and still tracks.

2

Prefer TRUNCATE over DELETE for full clears

If you must keep the table, TRUNCATE the whole thing rather than DELETE-ing all rows.

3

Build empty staging with WITH NO DATA

CREATE TEMP TABLE stage AS SELECT * FROM t WITH NO DATA; — schema only, no data movement.

4

Use ON COMMIT DROP in procedures

Procedure-scoped temp tables should self-clean at transaction end to avoid leftovers and lock contention.

Data Types & Expressions

Type consistency, materialised expression columns, and the cost of regex/JSON.

Be Consistent With Types

Implicit casts are invisible in the SQL but obvious in EXPLAIN (look for ::INT8, ::VARCHAR).

OFFENDING SQL

```
-- joining/comparing mismatched types
SELECT *
FROM orders o          -- cust_id INT8
JOIN cust c            -- cust_id INT4
  ON o.cust_id = c.cust_id;
-- c.cust_id::INT8 every row

WHERE order_no = '12345' -- VARCHAR
                  -- vs INT
```

Type mismatches inject per-row casts and can force a redistribution and bad estimates.

FIXED SQL

```
-- model join keys with identical types
-- (fix the DDL, or cast the constant)
SELECT *
FROM orders o
JOIN cust c
  ON o.cust_id = c.cust_id; -- INT8=INT8

WHERE order_no = 12345 -- match type
                      -- of the column
```

Identical key types -> no per-row cast, clean estimates, collocated join.

Takeaway: Standardise the types of join/filter keys across tables; never rely on implicit casting in hot paths.

Materialize Expressions; Mind Regex & JSON Cost

If an expression appears in your hot joins or filters, turn it into a stored column.

PROBLEM

- Joins/filters/grouping on expressions are very slow.
- Bad row estimates and full scans.
- Statements with many regex/JSON calls compile slowly and run hot.

WHY IT HAPPENS

- Expressions can't use range-restricted scans and are hard to estimate.
- They often force redistribution on both sides of a join.
- Regex and JSON functions generate large, expensive code; cost grows with how many appear.

HOW TO FIX

- Materialise common expressions as real columns at load (e.g. UPPER(), date parts, extracted codes). Some customers use dv_ or m_ for derived/materialized column names.
- Compute regex/JSON-heavy fields once into columns instead of per query.
- Minimise the number of regex/JSON functions per statement.

Skew Handling

NULL and low-cardinality skew — and the problem patterns behind most slow queries.

NULL Skew in Distribution / Join Keys

NULLs are the most common single source of both at-rest and runtime skew.

OFFENDING SQL

```
-- many NULLs in the dist/join key
-- all NULLs hash to one worker
SELECT ...
FROM fact f
JOIN dim d ON f.dim_id = d.dim_id;
-- f.dim_id is 60% NULL
```

All NULLs share one hash bucket/worker -> at-rest and runtime skew, long hash chains.

FIXED SQL

```
-- 1) split NULLs out with UNION ALL
-- 2) or DISTRIBUTE REPLICATE the dim
-- 3) or load a 'junk' non-NULL value
UPDATE staging
SET dim_id = (other_id % 1000) * -1
WHERE dim_id IS NULL; -- at ETL time
```

Spread or remove the NULLs so they no longer collapse onto a single worker.

Takeaway: Treat NULL in a distribution/join key as skew: UNION ALL it out, REPLICATE the other side, or substitute junk values at load.

Problem Patterns — A Quick Recap

95% of query problems come from missing the basics. Scan this list before deep-diving a plan.

1

Joins / filters on expressions

Disable skipping, force redistribution, ruin estimates

2

Distribution & runtime skew

Uneven data or work; one slow worker drags the query

3

Low-cardinality multi-column joins

Long hash chains; concatenate or add a synthetic key

4

NULL skew

NULLs collapse onto one worker

5

Aggregating on low-card cols / expressions

Few hash values; reorder or rewrite

6

Iterative instead of set-based

Row loops & singleton DML fight the engine

Planner Settings & Hinting

Last-resort levers — after the DDL and the SQL are already right.

Safe Settings & When to Reach for Them

Most planner GUCs are a slippery slope. Two are commonly safe; the rest, handle with care.

- **Fix root causes first** Distribution, missing constraints, skew, stale stats, and auto-analyze beat any hint.
- **assume_noop_function_estimation** Tells the planner TRIM/LOWER/UPPER don't change selectivity — safer estimates for those functions.
- **join_collapse_limit** Controls how many tables the planner reorders (default 8). Raise it to consider more orders; lower to 1 to lock your written order.
- **Prefer structural fixes** Synthetic distribution keys, views/CTEs, and explicit join ordering before hints.
- **Hinting is a double-edged sword** Powerful for generated SQL and unavoidable expression joins — but easy to misuse.

Apply scope

- System / user / session
- Per-query SETTING(...) clause
- Dynamically in a WLM rule
- SELECT-type statements only

Plan Hinting: Raising join_collapse_limit

EXPLAIN the statement before and after — confirm the order actually improved.

OFFENDING SQL

```
-- 12-table report; planner only
-- reorders the first 8 -> poor order
SELECT ...
FROM t1 JOIN t2 ... JOIN t12 ...;
-- giant intermediates, spills
```

With many joins the planner stops permuting at 8 tables and can lock in a bad order.

FIXED SQL

```
-- consider more candidate orders
SETTING (join_collapse_limit = 12)
SELECT ...
FROM t1 JOIN t2 ... JOIN t12 ...;

-- or set per-session / via a WLM
-- hinting rule (SELECT statements)
```

More candidate orders let the planner find a cheaper plan — at the cost of longer planning.

Takeaway: Raise `join_collapse_limit` only for the heavy multi-join statements that need it; planning time grows fast.

Query Investigation

A repeatable method beats guessing — change one thing, measure, repeat.

A Repeatable Tuning Method

95% of problems are the basics: distribution, ordering, types, expressions, locks, set-vs-row, WLM.

Method

- **Change one thing** iterate; never tune five things at once.
- **Start at the fact table** work joins first, then scans.
- **Decompose views** test against base tables to expose hidden costs.
- **Ignore first-run compile** the first execution includes code generation.
- **Clock time != work** check for queuing & concurrency.

What to Check

- **Spill** `io_spill_space_bytes_max > 0` is a 10x+ penalty.
- **read_efficiency** low % = a skipping / predicate problem.
- **Skew** actual rows per worker uneven.
- **Redistribute nodes** DISTRIBUTE / BROADCAST = network cost.
- **EXPLAIN (ANALYZE)** per-step breakdown; plan vs actual rows.

Plan vs Actual Rows: Make Them Close

EXPLAIN shows planned rows (the estimate); EXPLAIN (ANALYZE) adds actual rows. Find the node where they diverge most -- fix that estimate, re-measure.

Why a close match matters

- **The whole plan rides on the estimate** join order, join type, build side, memory, and redistribution are all chosen from rows_planned.
- **Join order** bad estimates reduce rows in the wrong sequence -> huge intermediates carried forward.
- **Memory & spill** an under-estimate sizes the hash too small -> the query spills (10x+).
- **Build side & hash chains** the wrong (large) side gets hashed -> long hash chains.
- **Needless redistribution** a wrong cardinality triggers an avoidable DISTRIBUTE / BROADCAST.
- **Close = trust the plan** a small gap means the chosen plan is sound; a 10x+ gap is the root cause to chase.

What to do about a gap

- **ANALYZE** refresh statistics after a big load or heavy change — often the entire fix.
- **Add constraints** PK / UNIQUE / FK (informational, but the planner uses them to estimate and eliminate joins).
- **Materialize expressions** store UPPER()/LOWER()/date-parts as real columns so statistics exist.
- **Temp-table the messy set** push a CTE/subquery into CREATE TEMP TABLE AS — it gets its own stats.
- **Same key types** avoid implicit casts that defeat estimates on joins.
- **Last resort** hints / join_collapse_limit / OFFSET 0 only when estimates truly can't be fixed.

Reading EXPLAIN (ANALYZE) End-to-End

Let the metrics point to the single worst node — then fix that, not everything at once.

EXPLAIN (ANALYZE)

```
CTAS y
INNER LOOP JOIN ON true
|-SCAN rg1m AS r1  read_efficiency=100%
|-DISTRIBUTE REPLICATE
  EXPRESSION calculate: (NULL cast)
  SCAN rg1m AS r2

rows_planned  1.0e11
rows_actual   1.3e12  skew 0.00
Read: 0.6MiB  Network: 720MiB
Write: 14.7TiB  spill: 0
```

- **Plan vs actual rows** a huge gap (100B planned, 1.3T actual) signals a bad estimate — here an unintended cross/loop join.
- **read_efficiency 100%** the scans are fine; scanning is not the bottleneck.
- **DISTRIBUTE / Network** 720MiB moved across the network — redistribution is in play.
- **Write 14.7TiB** an enormous write betrays a runaway join emitting far too many rows.
- **The method** follow the biggest number to the offending node, fix that one thing, then re-measure.

13

Feature-Specific Performance

Newer data types bring their own query-writing tips: JSON and geospatial.

JSON vs JSONB

If you query JSON documents repeatedly, store them as JSONB to avoid re-parsing per call.

JSON (stored as text)

- **Stored verbatim** exact bytes as ingested.
- **Re-parsed each access** every function call re-parses the string.
- **Use when** write-once, rarely queried; need the original text.

JSONB (indexed binary)

- **Validated & canonicalised** whitespace removed, keys ordered at insert.
- **No re-parsing** metadata speeds key / index lookups.
- **Use when** you query the same documents repeatedly.

Ten Habits of Efficient Yellowbrick SQL

If you remember nothing else, remember these.

1

Bare column, constant predicate

so scans can skip data

2

Join on the distribution key

collocate; avoid redistribution

3

Filter early, order joins

most selective first

4

Same types, no expressions in keys

materialise expressions

5

Group on the distribution column

high-cardinality first

6

Rewrite COUNT(DISTINCT)

group-then-count

7

CTE+OFFSET 0 / temp tables

control order, reuse, stats

8

Set-based DML, never singleton

stage then one statement

9

DROP > TRUNCATE > DELETE

and recreate for big changes

10

Read EXPLAIN & sys.query

fix what the plan shows



Thank You

Module 5b — Efficient SQL · Your Data Anywhere